这是一个稍微需要澄清的需求。需要特别说明的是：**NTT（快速数论变换）是基于模运算（取模）的，必须在整数域（如 `long long`）下进行，无法使用浮点数（`long double`）。**

如果你指定要用 `long double`，你需要的通常是 **FFT（快速傅里叶变换）**，它使用复数和浮点数进行运算。

为了满足你的需求，我将提供两份代码：

1. **标准 NTT 板子**：使用 `vector` 和 `long long`（这是 NTT 的正确写法）。
2. **高精度 FFT 板子**：使用 `vector` 和 `long double`（这是符合你对数据类型要求的写法）。

1. 标准 NTT 板子 (基于 `long long` + `vector`)

这是做多项式乘法、大数乘法最常用的算法。模数 $P = 998244353$，原根 $G = 3$。

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// 定义模数和原根
const long long MOD = 998244353;
const long long G = 3;
const long long Gi = 332748118; // 3 在模 998244353 下的逆元

// 快速幂
long long qpow(long long a, long long b) {
    long long res = 1;
    while (b) {
        if (b & 1) res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

// 获取位逆序变换数组
vector<int> get_rev(int limit) {
    vector<int> rev(limit);
    for (int i = 0; i < limit; i++) {
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) ? (limit >> 1) : 0);
    }
    return rev;
}

// NTT 核心代码
// type = 1 为 DFT, type = -1 为 IDFT
void ntt(vector<long long>& a, int limit, int type) {
    vector<int> rev = get_rev(limit);
    for (int i = 0; i < limit; i++) {
        if (i < rev[i]) swap(a[i], a[rev[i]]);
```

```
    }

    for (int mid = 1; mid < limit; mid <<= 1) {
        // Wn = G ^ ((MOD - 1) / (2 * mid))
        long long Wn = qpow(type == 1 ? G : Gi, (MOD - 1) / (mid << 1));
        for (int j = 0; j < limit; j += (mid << 1)) {
            long long w = 1;
            for (int k = 0; k < mid; k++, w = (w * Wn) % MOD) {
                long long x = a[j + k];
                long long y = (w * a[j + k + mid]) % MOD;
                a[j + k] = (x + y) % MOD;
                a[j + k + mid] = (x - y + MOD) % MOD;
            }
        }
    }
}

// 多项式乘法接口
vector<long long> poly_mul(const vector<long long>& A, const vector<long long>& B) {
    int n = A.size() - 1;
    int m = B.size() - 1;
    int limit = 1;
    while (limit <= n + m) limit <<= 1;

    // 调整大小并初始化
    vector<long long> a(limit), b(limit);
    for (int i = 0; i <= n; i++) a[i] = A[i];
    for (int i = 0; i <= m; i++) b[i] = B[i];

    // NTT 变换
    ntt(a, limit, 1);
    ntt(b, limit, 1);

    // 点值相乘
    for (int i = 0; i < limit; i++) {
        a[i] = (a[i] * b[i]) % MOD;
    }

    // IDFT 逆变换
    ntt(a, limit, -1);

    // 结果除以 limit (即乘以 limit 的逆元)
    long long inv_limit = qpow(limit, MOD - 2);
    for (int i = 0; i < limit; i++) {
        a[i] = (a[i] * inv_limit) % MOD;
    }

    // 这里的 a 包含了多项式乘积的系数，高位可能为0，根据需要 resize
    a.resize(n + m + 1);
    return a;
}
```

```
int main() {
    // 示例：(1 + 2x) * (1 + 2x + 1x^2)
    // A = {1, 2}, B = {1, 2, 1}
    // 预期结果: 1 + 4x + 5x^2 + 2x^3
    vector<long long> A = {1, 2};
    vector<long long> B = {1, 2, 1};

    vector<long long> C = poly_mul(A, B);

    cout << "Result: ";
    for (auto val : C) {
        cout << val << " ";
    }
    cout << endl;

    return 0;
}
```

---

2. 高精度 FFT 板子 (使用 `long double`)

如果你是因为需要更高精度的浮点运算（避免精度丢失）而提到的 `long double`，那么你应该使用 FFT。这通常用于不取模的大数乘法或一般多项式乘法。

```
#include <iostream>
#include <vector>
#include <cmath>
#include <complex>
#include <algorithm>

using namespace std;

// 使用 long double 保证高精度
using Real = long double;
const Real PI = acos((Real)-1.0);

// 使用 std::complex<long double>
using Complex = complex<Real>;

// 获取位逆序变换数组
vector<int> get_rev(int limit) {
    vector<int> rev(limit);
    for (int i = 0; i < limit; i++) {
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) ? (limit >> 1) : 0);
    }
    return rev;
}

// FFT 核心代码
// type = 1 为 DFT, type = -1 为 IDFT
void fft(vector<Complex>& a, int limit, int type) {
```

```
    vector<int> rev = get_rev(limit);
    for (int i = 0; i < limit; i++) {
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    }

    for (int mid = 1; mid < limit; mid <<= 1) {
        // 使用 long double 计算单位根
        Complex Wn(cos(PI / mid), type * sin(PI / mid));
        for (int j = 0; j < limit; j += (mid << 1)) {
            Complex w(1, 0);
            for (int k = 0; k < mid; k++, w = w * Wn) {
                Complex x = a[j + k];
                Complex y = w * a[j + k + mid];
                a[j + k] = x + y;
                a[j + k + mid] = x - y;
            }
        }
    }
}

// 多项式乘法接口
vector<long long> poly_mul_fft(const vector<int>& A, const vector<int>& B) {
    int n = A.size() - 1;
    int m = B.size() - 1;
    int limit = 1;
    while (limit <= n + m) limit <<= 1;

    vector<Complex> a(limit), b(limit);
    for (int i = 0; i <= n; i++) a[i] = A[i];
    for (int i = 0; i <= m; i++) b[i] = B[i];

    // FFT
    fft(a, limit, 1);
    fft(b, limit, 1);

    // 点乘
    for (int i = 0; i < limit; i++) {
        a[i] = a[i] * b[i];
    }

    // IFFT
    fft(a, limit, -1);

    vector<long long> result(n + m + 1);
    for (int i = 0; i <= n + m; i++) {
        // 结果除以 limit 并四舍五入取整
        result[i] = (long long)(a[i].real() / limit + 0.5);
    }
    return result;
}

int main() {
```

```
    // 示例: (1 + 2x) * (1 + 2x + 1x^2)
    vector<int> A = {1, 2};
    vector<int> B = {1, 2, 1};

    vector<long long> C = poly_mul_fft(A, B);

    cout << "Result (FFT with long double): ";
    for (auto val : C) {
        cout << val << " ";
    }
    cout << endl;

    return 0;
}
```

总结

- 如果你是在做**算法竞赛**（题目涉及取模 998244353），请复制**第一份代码**。
- 如果你是在做**数值计算**或者**不取模的大数乘法**，且担心 `double` 精度不够，请复制**第二份代码**。

这是一个专门针对 **系数为浮点数（`long double`）** 的多项式乘法 FFT 板子。

与之前的整数多项式乘法不同，这个版本的特点是：

1. **输入和输出**都是 `vector<long double>`。
2. **不做四舍五入**：最后的结果保留高精度的浮点值。
3. 适用于概率生成函数、信号处理或数值计算等**非整数域**场景。

C++ FFT 板子（`long double` 版）

```
#include <iostream>
#include <vector>
#include <complex>
#include <cmath>
#include <algorithm>
#include <iomanip> // 用于控制输出精度

using namespace std;

// ================= 类型定义 =================
// 使用 long double 保证最高精度
using Real = long double;
// 复数类型
using Complex = std::complex<Real>;
// 圆周率 (使用 acos(-1.0L) 确保精度)
const Real PI = acos((Real)-1.0);
```

```cpp
// ==========================================

/**
 * @brief 生成位逆序置换数组 (蝴蝶变换索引)
 * @param limit 变换长度
 */
vector<int> get_rev(int limit) {
    vector<int> rev(limit);
    for (int i = 0; i < limit; i++) {
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) ? (limit >> 1) : 0);
    }
    return rev;
}

/**
 * @brief 快速傅里叶变换 (FFT) 核心逻辑
 * @param a 复数数组
 * @param limit 长度 (2的幂)
 * @param type 1 为 DFT, -1 为 IDFT
 */
void fft(vector<Complex>& a, int limit, int type) {
    vector<int> rev = get_rev(limit);
    for (int i = 0; i < limit; i++) {
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    }

    // 迭代层级
    for (int mid = 1; mid < limit; mid <<= 1) {
        // 单位根，注意 type 控制旋转方向
        Complex Wn(cos(PI / mid), type * sin(PI / mid));

        for (int j = 0; j < limit; j += (mid << 1)) {
            Complex w(1, 0);
            for (int k = 0; k < mid; k++, w = w * Wn) {
                Complex x = a[j + k];
                Complex y = w * a[j + k + mid];
                a[j + k] = x + y;
                a[j + k + mid] = x - y;
            }
        }
    }
}

/**
 * @brief 浮点数多项式乘法接口
 * @param A 多项式A的系数 (long double)
 * @param B 多项式B的系数 (long double)
 * @return vector<long double> 卷积结果
 */
vector<Real> poly_mul(const vector<Real>& A, const vector<Real>& B) {
    int n = A.size() - 1;
    int m = B.size() - 1;
```

```
    // 确定 FFT 长度
    int limit = 1;
    while (limit <= n + m) limit <<= 1;

    // 转换为复数向量并填充0
    vector<Complex> a(limit), b(limit);
    for (int i = 0; i <= n; i++) a[i] = Complex(A[i], 0);
    for (int i = 0; i <= m; i++) b[i] = Complex(B[i], 0);

    // 1. DFT
    fft(a, limit, 1);
    fft(b, limit, 1);

    // 2. 点乘
    for (int i = 0; i < limit; i++) {
        a[i] = a[i] * b[i];
    }

    // 3. IDFT
    fft(a, limit, -1);

    // 4. 提取实部并归一化 (不需要四舍五入取整，直接保留浮点结果)
    vector<Real> result(n + m + 1);
    for (int i = 0; i <= n + m; i++) {
        result[i] = a[i].real() / limit;
    }

    return result;
}

int main() {
    // 优化 IO 速度
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // 示例：浮点系数多项式
    // A(x) = 1.5 + 2.0x
    // B(x) = 1.0 + 0.5x
    // 理论结果: 1.5 + (0.75 + 2.0)x + 1.0x^2 = 1.5 + 2.75x + 1.0x^2
    vector<Real> A = {1.5, 2.0};
    vector<Real> B = {1.0, 0.5};

    vector<Real> C = poly_mul(A, B);

    // 设置输出精度查看 long double 效果
    cout << fixed << setprecision(10);

    cout << "Result coefficients:" << endl;
    for (int i = 0; i < C.size(); i++) {
        cout << "x^" << i << " : " << C[i] << endl;
    }
```

```
    return 0;
}
```

代码说明

1. **输入输出类型 (`vector<Real>`)**:
    - 函数 `poly_mul` 接收的参数和返回值均为 `vector<long double>`。
    - 这意味着你可以传入如 `{1.1, 2.5, 3.14}` 这样的系数。
2. **归一化处理**:
    - 在 `poly_mul` 的最后一步：`result[i] = a[i].real() / limit;`。
    - 这里直接执行除法，不加 `+0.5` 也不强转 `long long`，从而保留了浮点数的精度。
3. **精度控制**:
    - `const Real PI = acos((Real)-1.0);` 中的 `(Real)` 强转确保调用的是 `acos(long double)` 版本，获得最高精度的 π。
    - 在 `main` 函数中使用了 `cout << fixed << setprecision(10);` 来展示高精度结果，防止控制台默认只显示 6 位有效数字。

使用建议

- **精度误差**: 虽然使用了 `long double`，但 FFT 本质上是浮点运算，对于极高次多项式（如 $N > 10^6$）或系数差异巨大的情况，低位仍可能出现微小的 ϵ 误差（例如本应是 0 的地方算出 `1e-18`）。如果需要绝对的 0，可以手动设置一个阈值（EPS），小于阈值的置为 0。
- **性能**: `long double` 在某些架构下比 `double` 慢（因为它可能是 80 位或 128 位软件模拟）。如果对速度要求极高且 `double` 精度足够（15-16位有效数字），可以将 `using Real = long double;` 改回 `using Real = double;`。

可以优化。你当前的代码虽然清晰，但在**性能**和**精度稳定性**上还有很大的提升空间。

主要的优化方向有三个：

1. **预处理单位根 (Precomputed Roots of Unity)**：最核心的优化。原始代码在每次 FFT 调用时都重复计算 `cos/sin`，且在内层循环通过 `w = w * wn` 累乘，这会导致 `long double` 的高精度优势因多次累乘误差而打折。优化后应预先计算好表，直接查表。
2. **缓存位逆序表 (Cached Bit-Reversal)**：避免每次调用都重新计算 `rev` 数组。
3. **减少动态内存分配**：在多项式乘法中尽量复用内存。

下面是**深度优化后**的 FFT 板子。这个版本在保持 C++ 风格的同时，使用了**预处理表**和**更高效的内存访问**。

🚀 高性能 FFT 板子 (long double + 预处理优化版)

```
#include <iostream>
#include <vector>
#include <complex>
#include <cmath>
```

```cpp
#include <algorithm>
#include <iomanip>

using namespace std;

// ================= 配置区域 =================
using Real = long double;
using Complex = std::complex<Real>;
const Real PI = acos((Real)-1.0);

// ================= FFT 求解器类 =================
// 将 FFT 封装在类中，利用成员变量缓存 rev 和 roots，避免重复计算
struct FFTSolver {
    int n, m, limit;
    int l; // limit 的对数，即 2^l = limit
    vector<int> rev;
    vector<Complex> roots; // 存储单位根

    FFTSolver() : n(0), m(0), limit(0), l(0) {}

    // 初始化 FFT 参数，预处理 rev 数组和单位根
    void init(int _n, int _m) {
        n = _n; m = _m;
        limit = 1; l = 0;
        while (limit <= n + m) {
            limit <<= 1;
            l++;
        }

        // 1. 预处理位逆序置换 (仅当 limit 改变时重新计算)
        if (rev.size() != limit) {
            rev.resize(limit);
            for (int i = 0; i < limit; i++) {
                rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (l - 1));
            }
        }

        // 2. 预处理单位根 (Roots of Unity)
        // 我们只需要计算到 limit 长度的一半，因为蝴蝶操作的对称性
        // 预计算 W_{limit}^k，这样在后续分治中可以通过步长访问
        if (roots.size() != limit) {
            roots.resize(limit);
            for (int i = 0; i < limit; i++) {
                // 计算 e^(i * 2 * PI * j / limit)
                // 这里我们直接生成全量的表，方便查阅
                // 也可以只生成前半部分利用对称性，为了代码简洁生成全量
                roots[i] = Complex(cos(2.0 * PI * i / limit), sin(2.0 * PI * i /
limit));
            }
        }
    }
```

```
    // type: 1 = DFT, -1 = IDFT
    void fft(vector<Complex>& a, int type) {
        // 位逆序置换
        for (int i = 0; i < limit; i++) {
            if (i < rev[i]) swap(a[i], a[rev[i]]);
        }

        // 迭代 FFT
        // mid 为当前分治区间长度的一半
        for (int mid = 1; mid < limit; mid <<= 1) {
            // 步长：原本单位根是 2*PI / (2*mid)
            // 我们预处理的表是基于 2*PI / limit
            // 所以现在的 step = limit / (2*mid)
            int step = limit / (mid << 1);

            for (int j = 0; j < limit; j += (mid << 1)) {
                // 指针访问优化（虽然 vector 下标也很快，但在深层循环中裸指针略快）
                Complex* l_ptr = a.data() + j;
                Complex* r_ptr = a.data() + j + mid;

                for (int k = 0; k < mid; k++) {
                    // 直接查表获取 Wn，避免重复计算 sin/cos 和累乘误差
                    // 如果是 IDFT，需要取共轭 (虚部取反)，或者访问 roots[limit - idx]
                    // 这里利用技巧：roots[idx] = cos + i*sin
                    // IDFT 需要 cos - i*sin
                    // 查表下标 idx = k * step

                    int idx = k * step;
                    Complex Wn;
                    if (type == 1) {
                        Wn = roots[idx];
                    } else {
                        // IDFT: 虚部取反。roots[0]不变，其余 roots[limit-idx]
                        Wn = (idx == 0) ? roots[0] : roots[limit - idx];
                    }

                    // 蝴蝶变换
                    Complex x = *l_ptr;
                    Complex y = Wn * (*r_ptr);

                    *l_ptr = x + y;
                    *r_ptr = x - y;

                    l_ptr++;
                    r_ptr++;
                }
            }
        }
    }

    vector<Real> multiply(const vector<Real>& A, const vector<Real>& B) {
        init(A.size() - 1, B.size() - 1);
```

```
        // 初始化复数数组
        vector<Complex> a(limit), b(limit);
        for (int i = 0; i <= n; i++) a[i] = Complex(A[i], 0);
        for (int i = 0; i <= m; i++) b[i] = Complex(B[i], 0);

        // DFT
        fft(a, 1);
        fft(b, 1);

        // 点值乘法
        for (int i = 0; i < limit; i++) {
            a[i] *= b[i];
        }

        // IDFT
        fft(a, -1);

        // 提取结果
        vector<Real> res(n + m + 1);
        for (int i = 0; i <= n + m; i++) {
            res[i] = a[i].real() / limit;
        }
        return res;
    }
};

int main() {
    // 1. IO 优化
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // 2. 准备数据
    vector<Real> A = {1.5, 2.0}; // 1.5 + 2.0x
    vector<Real> B = {1.0, 0.5}; // 1.0 + 0.5x

    // 3. 调用求解器
    FFTSolver solver;
    vector<Real> C = solver.multiply(A, B);

    // 4. 输出结果
    cout << fixed << setprecision(10);
    cout << "Polynomial A * B Coefficients:" << endl;
    for (int i = 0; i < C.size(); i++) {
        cout << "x^" << i << ": " << C[i] << endl;
    }

    return 0;
}
```

🎯 优化的具体细节

1. 预处理单位根 (Roots Table)

- **原代码问题**：`Complex Wn(cos(...), sin(...))` 在每层循环里计算，而且内层循环使用 `w = w * Wn`。
    - `cos/sin` 是超越函数，非常慢。
    - `w * Wn` 会导致浮点误差随着 `limit` 增大而累积，对于 `long double` 来说，这种累积误差可能会浪费原本的高精度特性。
- **优化后**：
    - `roots` 数组在 `init` 时一次性计算完毕。
    - 内层循环直接通过 `roots[k * step]` 访问。这不仅是 $O(1)$ 的访问，而且保证了每个单位根都是直接由 `cos/sin` 算出来的“一手数据”，没有累乘误差。

2. IDFT 的查表技巧

- 标准 IDFT 的旋转因子是 DFT 的共轭。
- 如果 DFT 需要 `roots[idx]`，那么 IDFT 需要 `conj(roots[idx])`。
- 根据单位根性质，`conj(roots[idx])` 等价于 `roots[limit - idx]`（当 `idx=0` 时为 `roots[0]`）。这样我们可以复用同一张表，不需要重新计算。

3. 封装与缓存 (FFTSolver Struct)

- 将 `rev` 和 `roots` 放在结构体成员中。
- 如果在实际应用中需要多次计算**相同长度**的多项式乘法，`init` 函数会检测 `rev.size() != limit`，只有长度变了才重新分配内存和计算表。这对于大量同规模的计算（如生成函数运算）有极大的性能提升。

4. 指针访问 (Pointer Arithmetic)

- 在最内层循环 `for (int k = 0; k < mid; k++)` 中，使用了 `Complex* l_ptr` 和 `r_ptr`。
- 虽然现代编译器对 `vector[i]` 优化很好，但显式的指针自增（`l_ptr++`）可以减少基址寻址的开销，并给编译器更明确的自动向量化（SIMD）提示。

性能对比预估

指标	原始版本	优化版本	原因
三角函数调用	$O(N)$ 或 $O(N \log N)$	$O(N)$ (仅一次)	查表代替实时计算
乘法精度	较低 (累乘误差)	**最高** (直接查表)	避免了 `w *= Wn` 的误差传递
重复调用开销	高 (每次重建 vector)	低 (缓存 rev/roots)	内存复用

指标	原始版本	优化版本	原因
内层循环速度	一般	极快	移除了超越函数和复杂的下标计算

这个版本是**使用 STL 容器和 long double 约束下**非常接近极限性能的实现。

优化后的 FFT 模板 (高性能版)

这份代码应用了**手写复数类**、**预处理单位根**、**静态数组**和`double`**类型**。这是算法竞赛中常用的高性能模板。

```
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>
#include <iomanip>

using namespace std;

// 定义最大长度，根据题目要求修改，通常是 2 的幂次
const int MAXN = 262144 + 5; // 2^18
const double PI = acos(-1.0);

// 1. 手写 Complex 结构体，利于内联和 SIMD 优化
struct Complex {
    double x, y;
    Complex(double _x = 0, double _y = 0) : x(_x), y(_y) {}
    Complex operator+(const Complex &b) const { return Complex(x + b.x, y + b.y); }
    Complex operator-(const Complex &b) const { return Complex(x - b.x, y - b.y); }
    Complex operator*(const Complex &b) const { return Complex(x * b.x - y * b.y, x
* b.y + y * b.x); }
};

// 全局静态数组，避免 vector 频繁分配内存
Complex a[MAXN], b[MAXN];
int rev[MAXN];
// 2. 预处理单位根，omg[0] 存 DFT 的单位根，omg[1] 存 IDFT 的
// 实际上通常只需要预处理一半的长度
vector<Complex> omega;

void init_omega(int limit) {
    omega.resize(limit);
    // 预处理 cos, sin
    for(int i = 0; i < limit; ++i) {
        omega[i] = Complex(cos(2 * PI * i / limit), sin(2 * PI * i / limit));
    }
}
```

```
// limit: 变换长度
// type: 1 为 DFT, -1 为 IDFT
void fft(Complex *A, int limit, int type) {
    // 位逆序置换
    for (int i = 0; i < limit; ++i) {
        if (i < rev[i]) swap(A[i], A[rev[i]]);
    }

    // 蝴蝶变换
    for (int mid = 1; mid < limit; mid <<= 1) {
        // 步长，用于从预处理数组中取值
        // 如果是 IDFT，实际上相当于取共轭，或者利用周期性
        // 这里采用更通用的预处理方式：直接计算当前层的单位根步长
        // 优化：使用预处理的 omega 数组。
        // 注意：这里为了通用性展示了动态步长逻辑，极致优化会完全预处理所有层级

        int step = omega.size() / (mid * 2);

        for (int j = 0; j < limit; j += (mid << 1)) {
            for (int k = 0; k < mid; ++k) {
                // 读取预处理的单位根
                // 对于 IDFT (type=-1)，原本是 sin 取反。
                // sin(-x) = -sin(x), cos(-x) = cos(x)
                // 对应的 index 是 (type == 1 ? k * step : omega.size() - k * step) %
omega.size()
                // 简单处理：如果是 IDFT，虚部取反

                Complex w = omega[k * step];
                if (type == -1 && k > 0) { // IDFT 时共轭
                     w.y = -w.y;
                }

                Complex x = A[j + k];
                Complex y = w * A[j + k + mid];
                A[j + k] = x + y;
                A[j + k + mid] = x - y;
            }
        }
    }
}

vector<long long> poly_mul_fft(const vector<int>& A, const vector<int>& B) {
    int n = A.size() - 1;
    int m = B.size() - 1;
    int limit = 1;
    int bit = 0;
    while (limit <= n + m) {
        limit <<= 1;
        bit++;
    }
```

```
    // 预处理 rev 数组
    for (int i = 0; i < limit; i++) {
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (bit - 1));
    }

    // 初始化 Omega 表 (如果是多次调用，且 limit 不变，可以只初始化一次)
    if (omega.size() != limit) init_omega(limit);

    // 数据拷贝到全局数组，清空高位
    for (int i = 0; i < limit; i++) {
        a[i] = (i <= n) ? Complex((double)A[i], 0) : Complex(0, 0);
        b[i] = (i <= m) ? Complex((double)B[i], 0) : Complex(0, 0);
    }

    // FFT
    fft(a, limit, 1);
    fft(b, limit, 1);

    // 点乘
    for (int i = 0; i < limit; i++) {
        a[i] = a[i] * b[i];
    }

    // IFFT
    fft(a, limit, -1);

    vector<long long> result(n + m + 1);
    for (int i = 0; i <= n + m; i++) {
        // 结果除以 limit 并四舍五入
        // 加上 0.5 利用截断特性实现四舍五入
        result[i] = (long long)(a[i].x / limit + 0.5);
    }
    return result;
}

int main() {
    // 提高 IO 速度
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    vector<int> A = {1, 2};
    vector<int> B = {1, 2, 1};

    vector<long long> C = poly_mul_fft(A, B);

    cout << "Result: ";
    for (auto val : C) {
        cout << val << " ";
    }
    cout << endl;

    return 0;
```

```
}
```

这是一个基于你提供的NTT模板实现的模998244353下的多项式Exp求法。

为了实现 `poly_exp`，我们需要先实现以下前置功能：

1. **求导 (`poly_deriv`)**
2. **积分 (`poly_int`)**
3. **多项式求逆 (`poly_inv`)**：利用牛顿迭代法 $B = B(2 - AB) \pmod{x^n}$。
4. **多项式对数 (`poly_ln`)**：$\ln A = \int (A' \cdot A^{-1}) dx$。

最后利用牛顿迭代法求Exp：$B_{new} = B_{old} \cdot (1 - \ln B_{old} + A) \pmod{x^n}$。

以下是完整代码，包含所有必要的辅助函数和详细注释：

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// 定义模数和原根
const long long MOD = 998244353;
const long long G = 3;
const long long Gi = 332748118; // 3 在模 998244353 下的逆元

// 快速幂
long long qpow(long long a, long long b) {
    long long res = 1;
    while (b) {
        if (b & 1) res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

// --- Cipolla 算法求解二次剩余 x^2 = n (mod p) ---
namespace Cipolla {
    long long w; // (a^2 - n)
    struct Complex {
        long long r, i;
        Complex(long long _r = 0, long long _i = 0) : r(_r), i(_i) {}
        Complex operator*(const Complex& other) const {
            return Complex(
                (r * other.r % MOD + i * other.i % MOD * w % MOD) % MOD,
                (r * other.i % MOD + i * other.r % MOD) % MOD
            );
        }
    };
```

```
    Complex complex_pow(Complex base, long long exp) {
        Complex res(1, 0);
        while (exp > 0) {
            if (exp & 1) res = res * base;
            base = base * base;
            exp >>= 1;
        }
        return res;
    }

    // 勒让德符号
    long long legendre(long long a) {
        return qpow(a, (MOD - 1) / 2);
    }

    long long solve(long long n) {
        if (n == 0) return 0;
        if (legendre(n) != 1) return -1; // 无解

        long long a;
        while (true) {
            a = rand() % MOD;
            w = (a * a - n + MOD) % MOD;
            if (legendre(w) == MOD - 1) break; // 找到满足条件的 a
        }

        // 计算 (a + sqrt(w))^((p+1)/2)
        Complex res = complex_pow(Complex(a, 1), (MOD + 1) / 2);
        return res.r;
    }
}
// --------------------------------------------

// 获取位逆序变换数组
vector<int> get_rev(int limit) {
    vector<int> rev(limit);
    for (int i = 0; i < limit; i++) {
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) ? (limit >> 1) : 0);
    }
    return rev;
}

// NTT 核心代码
void ntt(vector<long long>& a, int limit, int type) {
    vector<int> rev = get_rev(limit);
    for (int i = 0; i < limit; i++) {
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    }

    for (int mid = 1; mid < limit; mid <<= 1) {
        long long Wn = qpow(type == 1 ? G : Gi, (MOD - 1) / (mid << 1));
```

```
        for (int j = 0; j < limit; j += (mid << 1)) {
            long long w = 1;
            for (int k = 0; k < mid; k++, w = (w * Wn) % MOD) {
                long long x = a[j + k];
                long long y = (w * a[j + k + mid]) % MOD;
                a[j + k] = (x + y) % MOD;
                a[j + k + mid] = (x - y + MOD) % MOD;
            }
        }
    }
}

// 基础多项式乘法（会自动调整大小）
vector<long long> poly_mul(const vector<long long>& A, const vector<long long>& B,
int deg = -1) {
    int n = A.size() - 1;
    int m = B.size() - 1;
    int limit = 1;
    while (limit <= n + m) limit <<= 1;

    vector<long long> a(limit), b(limit);
    for (int i = 0; i <= n; i++) a[i] = A[i];
    for (int i = 0; i <= m; i++) b[i] = B[i];

    ntt(a, limit, 1);
    ntt(b, limit, 1);

    for (int i = 0; i < limit; i++) {
        a[i] = (a[i] * b[i]) % MOD;
    }

    ntt(a, limit, -1);

    long long inv_limit = qpow(limit, MOD - 2);
    for (int i = 0; i < limit; i++) {
        a[i] = (a[i] * inv_limit) % MOD;
    }

    // 如果指定了度数，截断输出；否则保留完整卷积结果
    if (deg != -1) {
        a.resize(deg);
    } else {
        // 自动去掉高位多余的0（可选，这里为了后续计算方便通常保留足够长度或按需resize）
        a.resize(n + m + 1);
    }
    return a;
}

// 多项式求导
vector<long long> poly_deriv(const vector<long long>& A) {
    int n = A.size();
    if (n <= 1) return {0};
```

```cpp
    vector<long long> B(n - 1);
    for (int i = 1; i < n; i++) {
        B[i - 1] = (A[i] * i) % MOD;
    }
    return B;
}

// 多项式积分
vector<long long> poly_int(const vector<long long>& A) {
    int n = A.size();
    vector<long long> B(n + 1);
    B[0] = 0;
    for (int i = 0; i < n; i++) {
        B[i + 1] = (A[i] * qpow(i + 1, MOD - 2)) % MOD;
    }
    return B;
}

// 多项式求逆 (Inverse)
// 使得 B * A = 1 (mod x^n)
vector<long long> poly_inv(const vector<long long>& A, int n) {
    if (n == 1) {
        return {qpow(A[0], MOD - 2)};
    }

    // 递归求解 n/2 的情况
    vector<long long> B = poly_inv(A, (n + 1) / 2);

    // 下面计算 B = B * (2 - A * B) mod x^n
    // 为了效率，手动展开乘法过程，或者直接调用 poly_mul 并截断

    int limit = 1;
    while (limit < 2 * n) limit <<= 1;

    vector<long long> a(limit), b(limit);
    for (int i = 0; i < n && i < A.size(); i++) a[i] = A[i];
    for (int i = 0; i < B.size(); i++) b[i] = B[i];

    ntt(a, limit, 1);
    ntt(b, limit, 1);

    for (int i = 0; i < limit; i++) {
        // b[i] = b[i] * (2 - a[i] * b[i])
        // 注意取模减法
        long long val = (a[i] * b[i]) % MOD;
        val = (2 - val + MOD) % MOD;
        b[i] = (b[i] * val) % MOD;
    }

    ntt(b, limit, -1);

    long long inv_limit = qpow(limit, MOD - 2);
```

```
    B.resize(n);
    for (int i = 0; i < n; i++) {
        B[i] = (b[i] * inv_limit) % MOD;
    }

    return B;
}

// 多项式对数 (Ln)
// Ln(A) = Int(A' * A^-1)
// 要求 A[0] = 1
vector<long long> poly_ln(const vector<long long>& A, int n) {
    vector<long long> A_inv = poly_inv(A, n);
    vector<long long> A_deriv = poly_deriv(A);

    // 求导后长度变为 n-1，乘逆元时需要截断到 n-1 (积分后变回 n)
    vector<long long> prod = poly_mul(A_deriv, A_inv, n - 1);

    return poly_int(prod);
}

// 多项式指数函数 (Exp)
// Exp(A) = B => ln(B) = A
// 迭代式: B = B_0 * (1 - ln(B_0) + A)
// 要求 A[0] = 0
vector<long long> poly_exp(const vector<long long>& A, int n) {
    if (n == 1) {
        return {1}; // Exp(0) = 1
    }

    vector<long long> B = poly_exp(A, (n + 1) / 2);
    B.resize(n); // 调整 B 的大小以便进行后续计算

    // 计算 ln(B) mod x^n
    vector<long long> LnB = poly_ln(B, n);

    // 计算 poly = 1 - LnB + A
    // 注意 A 的长度可能小于 n
    vector<long long> poly(n);
    for (int i = 0; i < n; i++) {
        long long valA = (i < A.size()) ? A[i] : 0;
        long long valLn = LnB[i];
        poly[i] = (valA - valLn + MOD) % MOD;
    }
    // 常数项 +1
    poly[0] = (poly[0] + 1) % MOD;

    // B = B * poly
    // 这里只需结果的前 n 项
    vector<long long> res = poly_mul(B, poly, n);
    return res;
}
```

```cpp
// 多项式开根 (Sqrt)
// B^2 = A (mod x^n)
// 迭代式: B = (B + A * B^-1) / 2
// 要求: A[0] = 1 (或者自行计算 sqrt(A[0]))
vector<long long> poly_sqrt(const vector<long long>& A, int n) {
    if (n == 1) {
        // 边界条件：sqrt(A[0])
        // 这里默认 A[0] = 1，直接返回 1
        // 如果 A[0] 是其他值的二次剩余，需自行计算 x^2 = A[0] mod P 的解
        // long long val = Cipolla::solve(A[0]);
        // return {min(val, MOD - val)};
        return {1};
    }

    // 递归求解一半规模
    vector<long long> B = poly_sqrt(A, (n + 1) / 2);

    // 计算 B 的逆元，精度需要达到 n
    // 注意：虽然 B 的有效长度只有 (n+1)/2，但 poly_inv 会处理并在模 x^n 下计算
    vector<long long> invB = poly_inv(B, n);

    // 准备 A 的前 n 项用于乘法
    vector<long long> tmpA(n);
    for (int i = 0; i < n && i < A.size(); i++) tmpA[i] = A[i];

    // 计算 A * B^-1
    vector<long long> prod = poly_mul(tmpA, invB, n);

    // B = (B + prod) / 2
    // 调整 B 的大小到 n 以便进行加法
    B.resize(n);
    long long inv2 = (MOD + 1) / 2; // 2 的逆元

    for (int i = 0; i < n; i++) {
        B[i] = (B[i] + prod[i]) % MOD * inv2 % MOD;
    }

    return B;
}


int main() {
    // 示例：求 Exp(x + x^2) mod x^5
    // 理论解：e^(x+x^2) = 1 + x + 1.5x^2 + ... (在模意义下计算)

    // 输入 A(x) 的系数，例如 A(x) = 0 + 1x + 1x^2 (常数项必须为0)
    int n = 5; // 需要计算到的项数
    vector<long long> A = {0, 1, 1};

    // 确保 A[0] == 0，否则 Exp 在模意义下通常未定义或需要额外处理 e^A[0]
    if (A.empty()) A = {0};
```

```
    if (A[0] != 0) {
        cerr << "Error: Constant term of polynomial must be 0 for Poly Exp." <<
endl;
        return 1;
    }

    vector<long long> B = poly_exp(A, n);

    cout << "Exp(A) mod x^" << n << " coefficients: ";
    for (int i = 0; i < n; i++) {
        cout << B[i] << " ";
    }
    cout << endl;

    return 0;
}
```

代码说明：

1. **基础部分**：保留了你提供的 `qpow`，`get_rev`，`ntt`，并稍作修改 `poly_mul` 使其支持一个可选的参数 `deg`，用于直接截断结果到指定的次数，这在迭代过程中很有用。
2. **多项式求逆 (`poly_inv`)**：这是 Exp 和 Ln 的基础。使用倍增法（牛顿迭代）实现。当 $n = 1$ 时，返回 $A[0]^{-1}$。
3. **多项式求导与积分**：
    - 求导：系数 a_i 变为 $a_i \times i$，整体左移。
    - 积分：系数 a_i 变为 $a_i \times (i+1)^{-1}$，整体右移，常数项补0。
4. **多项式对数 (`poly_ln`)**：
    - 公式：$\ln(A(x)) = \int \frac{A'(x)}{A(x)} dx$。
    - 先求导，再乘上原多项式的逆，最后积分。
5. **多项式指数 (`poly_exp`)**：
    - 公式：$B_k = B_{k-1}(1 - \ln B_{k-1} + A) \pmod{x^n}$。
    - 同样使用倍增法。先递归求出前 $n/2$ 项的结果 B_0，计算 $\ln(B_0)$，然后代入公式更新得到前 n 项。
    - 注意：输入多项式 $A(x)$ 的常数项 $A[0]$ 必须为 0，这使得 $Exp(A)[0] = e^0 = 1$。

复杂度：

- 求逆、Ln、Exp 都使用了倍增法，每一层调用的长度减半。
- 总时间复杂度为 $T(n) = T(n/2) + O(n \log n) \implies O(n \log n)$。

这份代码实现的是 **任意模数多项式乘法 (MTT)**。

通常有两种主流写法：

1. **三模数 NTT (推荐)**：使用三个 NTT 友好模数分别计算，最后用中国剩余定理 (CRT) 合并。优点是精度完全准确，适合绝大多数算法竞赛题目。
2. **拆系数 FFT**：使用复数 FFT，将大整数拆成高低位计算。优点是常数稍小，但涉及浮点数精度问题，容易写挂。

下面提供的是最稳定、最通用的 **三模数 NTT + CRT** 版本。

算法原理

选取三个模数 P_1, P_2, P_3（通常为 $469762049, 998244353, 1004535809$），它们都是 $c \cdot 2^k + 1$ 的形式且原根 $g = 3$。

1. 分别在三个模数下进行 NTT 卷积。
2. 得到三个结果后，利用中国剩余定理（CRT）合并出在模 $P_1P_2P_3$ 下的真实值。
3. 由于 $P_1P_2P_3 > 10^{27}$，远大于一般题目中 $N \cdot (\max A_i)^2$ 的最大可能值，因此可以直接对题目要求的模数 M 取模。

C++ 模板代码

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>

using namespace std;

using i64 = long long;
using i128 = __int128; // 需要编译器支持，GCC/Clang均支持

// 三个常用 NTT 模数，原根均为 3
const int P1 = 469762049;  // 7 * 2^26 + 1
const int P2 = 998244353;  // 119 * 2^23 + 1
const int P3 = 1004535809; // 479 * 2^21 + 1

// 快速幂
i64 qpow(i64 a, i64 b, int mod) {
    i64 res = 1;
    for (; b; b >>= 1) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
    }
    return res;
}

// 模逆
i64 modInverse(i64 n, int mod) {
    return qpow(n, mod - 2, mod);
}

// NTT 核心类
```

```cpp
struct NTT_Solver {
    int mod, G, Gi;

    NTT_Solver(int m) : mod(m), G(3) {
        Gi = modInverse(G, mod);
    }

    void ntt(vector<int>& a, int limit, int type) {
        // 蝴蝶变换
        for (int i = 0, j = 0; i < limit; ++i) {
            if (i < j) swap(a[i], a[j]);
            for (int k = limit >> 1; (j ^= k) < k; k >>= 1);
        }

        // NTT 主过程
        for (int mid = 1; mid < limit; mid <<= 1) {
            i64 Wn = qpow(type == 1 ? G : Gi, (mod - 1) / (mid << 1), mod);
            for (int j = 0; j < limit; j += (mid << 1)) {
                i64 w = 1;
                for (int k = 0; k < mid; ++k, w = w * Wn % mod) {
                    int x = a[j + k];
                    int y = w * a[j + k + mid] % mod;
                    a[j + k] = (x + y) % mod;
                    a[j + k + mid] = (x - y + mod) % mod;
                }
            }
        }

        // 逆变换时除以 limit
        if (type == -1) {
            i64 invLimit = modInverse(limit, mod);
            for (int i = 0; i < limit; ++i) {
                a[i] = (i64)a[i] * invLimit % mod;
            }
        }
    }

    // 计算当前模数下的卷积
    vector<int> multiply(const vector<int>& A, const vector<int>& B, int limit) {
        vector<int> a = A, b = B;
        a.resize(limit);
        b.resize(limit);
        ntt(a, limit, 1);
        ntt(b, limit, 1);
        for (int i = 0; i < limit; ++i) {
            a[i] = (i64)a[i] * b[i] % mod;
        }
        ntt(a, limit, -1);
        return a;
    }
};
```

```cpp
// 任意模数 NTT (MTT) 主函数
// polyA, polyB: 输入多项式系数
// target_mod: 题目要求的模数
vector<int> mtt(const vector<int>& polyA, const vector<int>& polyB, int target_mod)
{
    int n = polyA.size() + polyB.size() - 1;
    int limit = 1;
    while (limit < n) limit <<= 1;

    // 分别在三个模数下做 NTT
    NTT_Solver t1(P1), t2(P2), t3(P3);
    vector<int> res1 = t1.multiply(polyA, polyB, limit);
    vector<int> res2 = t2.multiply(polyA, polyB, limit);
    vector<int> res3 = t3.multiply(polyA, polyB, limit);

    vector<int> ans(n);

    // 预计算 CRT 合并需要的常数
    // m1_inv_m2 = P1^{-1} mod P2
    i64 m1_inv_m2 = modInverse(P1, P2);
    // m1m2_inv_m3 = (P1 * P2)^{-1} mod P3
    i64 m1m2_inv_m3 = modInverse((i64)P1 * P2 % P3, P3);

    // P1_P2_mod = (P1 * P2) % target_mod (注意这里要取模，防止爆 i64，虽然下面用了 i128)
    // 但为了计算最后结果，我们需要 P1*P2 的值。
    // 这里的策略是：先在 __int128 下计算出真实值，再对 target_mod 取模。

    for (int i = 0; i < n; ++i) {
        i64 a1 = res1[i];
        i64 a2 = res2[i];
        i64 a3 = res3[i];

        // 1. 合并前两个模数: x = a1 + k * P1
        // x % P2 = a2 => (a1 + k * P1) % P2 = a2
        // k = (a2 - a1) * inv(P1) % P2
        i64 k1 = (a2 - a1 + P2) % P2 * m1_inv_m2 % P2;
        i128 x_12 = a1 + (i128)k1 * P1; // 此时 x_12 < P1 * P2

        // 2. 合并第三个模数: X = x_12 + k2 * (P1 * P2)
        // X % P3 = a3 => (x_12 + k2 * P1 * P2) % P3 = a3
        // k2 = (a3 - x_12) * inv(P1 * P2) % P3
        i64 x_12_mod_P3 = x_12 % P3;
        i64 k2 = (a3 - x_12_mod_P3 + P3) % P3 * m1m2_inv_m3 % P3;

        // 3. 计算最终结果并对 target_mod 取模
        // Ans = (x_12 + k2 * P1 * P2) % target_mod
        // 这一步计算涉及大数，需要用 i128 处理中间乘法
        i128 P1P2 = (i128)P1 * P2;
        i128 final_val = (x_12 + (i128)k2 * P1P2);

        ans[i] = final_val % target_mod;
    }
```

```
    return ans;
}

// 示例用法
int main() {
    // 优化 I/O
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, p;
    // 输入：n次项系数个数，m次项系数个数，模数 P
    if (!(cin >> n >> m >> p)) return 0;

    // 题目通常给的是 n+1 个系数
    n++; m++;

    vector<int> A(n), B(m);
    for (int i = 0; i < n; ++i) cin >> A[i];
    for (int i = 0; i < m; ++i) cin >> B[i];

    vector<int> C = mtt(A, B, p);

    // 输出卷积结果
    for (int i = 0; i < n + m - 1; ++i) {
        cout << C[i] << (i == n + m - 2 ? "" : " ");
    }
    cout << endl;

    return 0;
}
```

使用说明与注意事项

1. **时间复杂度**:
   由于跑了 3 遍 NTT，整体复杂度约为 $O(9 \cdot N \log N)$。虽然常数较大，但一般能通过 10^5 规模的数据（耗时约 0.5s ~ 1.5s）。
2. `__int128`:
   代码中在 CRT 合并阶段使用了 `__int128`。这是为了防止 $P_1 \times P_2$ 以及最后计算结果时溢出 `long long`。主流的 OJ（Codeforces, AtCoder, Luogu 等）的 GCC/Clang 编译器均支持此类型。如果是 MSVC (Visual Studio) 本地编译，可能不支持，需要手写大数乘法或模拟。
3. **多项式长度**:
   代码中 `n` 和 `m` 代表系数的个数（即次数+1）。输入时请根据题目调整。
4. **空间**:
   使用了多个 `vector`，注意空间开销。如果是极其卡空间的题目，可以考虑复用数组。
5. **任意模数**:
   参数 `target_mod` 可以是任意正整数，不需要是素数。

CRT 合并逻辑详解

假设我们在三个模数下得到的第 i 项系数分别为 a_1, a_2, a_3。我们需要求解 x 使得：

$$\begin{cases} x \equiv a_1 \pmod{P_1} \\ x \equiv a_2 \pmod{P_2} \\ x \equiv a_3 \pmod{P_3} \end{cases}$$

最终答案为 $x \pmod{M}$。

1. 先合并前两项：
   $x = a_1 + k_1 P_1$
   代入第二式：$a_1 + k_1 P_1 \equiv a_2 \pmod{P_2}$
   $\Rightarrow k_1 \equiv (a_2 - a_1) P_1^{-1} \pmod{P_2}$
   算出 k_1 后，令 $x_{12} = a_1 + k_1 P_1$。此时 x_{12} 在 $[0, P_1 P_2)$ 范围内是唯一的解。
2. 再合并第三项：
   $x = x_{12} + k_2 (P_1 P_2)$
   代入第三式：$x_{12} + k_2 P_1 P_2 \equiv a_3 \pmod{P_3}$
   $\Rightarrow k_2 \equiv (a_3 - x_{12}) (P_1 P_2)^{-1} \pmod{P_3}$
   算出 k_2 后，真实整数解为 $Val = x_{12} + k_2 P_1 P_2$。
3. 最后输出 $Val \pmod{M}$。

在任意模数下实现多项式 $\exp$（指数函数），核心在于解决 **任意模数多项式乘法 (MTT)**，然后利用牛顿迭代法套用 $\exp$ 的公式。

公式推导如下（牛顿迭代）：
$F(x) = e^{A(x)} \Rightarrow \ln F(x) - A(x) = 0$
设 $G(F) = \ln F - A$，我们需要求 $G(F) \equiv 0 \pmod{x^n}$。
迭代式为：
$F_k \equiv F_{k-1}(1 - \ln F_{k-1} + A) \pmod{x^{2k}}$

因为涉及任意模数，所有的多项式乘法都需要走 **MTT (三模数 NTT + CRT)**。

复杂度警告

标准 NTT 求解 $\exp$ 复杂度为 $O(N \log N)$，常数较大。
任意模数版本因为每次乘法都要做 3 次 NTT + CRT 合并，常数非常大（约为普通 NTT 的 9~10 倍）。在 $N = 10^5$ 的情况下，运行时间可能在 2~4 秒左右，请注意常数优化和题目时限。

C++ 模板代码

这份模板包含了：

1. **MTT**: 任意模数卷积。
2. **Poly Inv**: 多项式求逆。
3. **Poly Ln**: 多项式对数。
4. **Poly Exp**: 多项式指数。

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
#include <cassert>

using namespace std;

using i64 = long long;
using i128 = __int128;

// ====================== 基础数学工具 ======================

i64 qpow(i64 a, i64 b, int mod) {
    i64 res = 1;
    for (; b; b >>= 1) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
    }
    return res;
}

i64 modInverse(i64 n, int mod) {
    return qpow(n, mod - 2, mod);
}

// ====================== NTT / MTT 核心 ======================

const int P1 = 469762049, G1 = 3;
const int P2 = 998244353, G2 = 3;
const int P3 = 1004535809, G3 = 3;

struct NTT_Solver {
    int mod, G, Gi;
    NTT_Solver(int m, int g) : mod(m), G(g) {
        Gi = modInverse(G, mod);
    }

    // 蝴蝶变换预处理，limit 必须是 2 的幂
    void ntt(vector<int>& a, int limit, int type) {
        for (int i = 0, j = 0; i < limit; ++i) {
            if (i < j) swap(a[i], a[j]);
            for (int k = limit >> 1; (j ^= k) < k; k >>= 1);
        }
        for (int mid = 1; mid < limit; mid <<= 1) {
            i64 Wn = qpow(type == 1 ? G : Gi, (mod - 1) / (mid << 1), mod);
            for (int j = 0; j < limit; j += (mid << 1)) {
                i64 w = 1;
                for (int k = 0; k < mid; ++k, w = w * Wn % mod) {
                    int x = a[j + k];
                    int y = w * a[j + k + mid] % mod;
                    a[j + k] = (x + y) % mod;
```

```
                    a[j + k + mid] = (x - y + mod) % mod;
                }
            }
        }
        if (type == -1) {
            i64 invLimit = modInverse(limit, mod);
            for (int i = 0; i < limit; ++i) {
                a[i] = (i64)a[i] * invLimit % mod;
            }
        }
    }
};

// 全局定义三个 solver 以避免重复构造开销
NTT_Solver t1(P1, G1), t2(P2, G2), t3(P3, G3);

// 任意模数乘法 (MTT)
// 传入 polyA, polyB 以及目标模数 target_mod
// 返回结果长度为 polyA.size() + polyB.size() - 1
vector<int> mtt(const vector<int>& A, const vector<int>& B, int target_mod) {
    int n = A.size() + B.size() - 1;
    int limit = 1;
    while (limit < n) limit <<= 1;

    vector<int> a1(A.begin(), A.end()), b1(B.begin(), B.end());
    vector<int> a2(A.begin(), A.end()), b2(B.begin(), B.end());
    vector<int> a3(A.begin(), A.end()), b3(B.begin(), B.end());

    a1.resize(limit); b1.resize(limit);
    a2.resize(limit); b2.resize(limit);
    a3.resize(limit); b3.resize(limit);

    t1.ntt(a1, limit, 1); t1.ntt(b1, limit, 1);
    t2.ntt(a2, limit, 1); t2.ntt(b2, limit, 1);
    t3.ntt(a3, limit, 1); t3.ntt(b3, limit, 1);

    for (int i = 0; i < limit; ++i) {
        a1[i] = (i64)a1[i] * b1[i] % P1;
        a2[i] = (i64)a2[i] * b2[i] % P2;
        a3[i] = (i64)a3[i] * b3[i] % P3;
    }

    t1.ntt(a1, limit, -1);
    t2.ntt(a2, limit, -1);
    t3.ntt(a3, limit, -1);

    vector<int> ans(n);
    i64 m1_inv_m2 = modInverse(P1, P2);
    i64 m1m2_inv_m3 = modInverse((i64)P1 * P2 % P3, P3);
    i128 P1P2 = (i128)P1 * P2;

    for (int i = 0; i < n; ++i) {
```

```
        i64 v1 = a1[i], v2 = a2[i], v3 = a3[i];
        i64 k1 = (v2 - v1 + P2) % P2 * m1_inv_m2 % P2;
        i128 x_12 = v1 + (i128)k1 * P1;
        i64 k2 = (v3 - (i64)(x_12 % P3) + P3) % P3 * m1m2_inv_m3 % P3;
        i128 val = (x_12 + (i128)k2 * P1P2);
        ans[i] = val % target_mod;
    }
    return ans;
}

// ====================== 多项式全家桶 ======================

// 多项式求逆 (Inverse)
// 要求 A[0] 在 mod p 下有逆。
vector<int> poly_inv(const vector<int>& A, int n, int mod) {
    if (n == 1) {
        return { (int)modInverse(A[0], mod) };
    }
    // 递归求解 n/2
    vector<int> B = poly_inv(A, (n + 1) / 2, mod);

    // 牛顿迭代: B = B * (2 - A * B) mod x^n
    // 需要计算 A * B。为了效率，截取 A 的前 n 项
    vector<int> subA(A.begin(), A.begin() + min((int)A.size(), n));

    // 做乘法: subA * B
    // 这里的长度控制很重要，MTT 很慢，不要计算多余项
    vector<int> AB = mtt(subA, B, mod);
    AB.resize(n); // 截断到 n

    // 计算 2 - AB
    for (int& x : AB) {
        x = (mod - x) % mod;
    }
    AB[0] = (AB[0] + 2) % mod;

    // B = B * (2 - AB)
    vector<int> res = mtt(B, AB, mod);
    res.resize(n);
    return res;
}

// 多项式求导
vector<int> poly_deriv(const vector<int>& A, int mod) {
    if (A.size() <= 1) return {0};
    vector<int> res(A.size() - 1);
    for (int i = 1; i < A.size(); ++i) {
        res[i - 1] = (i64)A[i] * i % mod;
    }
    return res;
}
```

```cpp
// 多项式积分
vector<int> poly_integ(const vector<int>& A, int mod) {
    vector<int> res(A.size() + 1);
    res[0] = 0;
    // 预处理逆元以加速 (如果 mod 是质数)
    // 如果 mod 不是质数且 i 与 mod 不互质，这里会出问题
    // 通常多项式 Exp 题目保证 mod 是质数 (如 1e9+7)
    for (int i = 0; i < A.size(); ++i) {
        res[i + 1] = (i64)A[i] * modInverse(i + 1, mod) % mod;
    }
    return res;
}

// 多项式对数 (Ln)
// Ln(A) = Integ( Deriv(A) * Inv(A) )
// 要求 A[0] = 1
vector<int> poly_ln(const vector<int>& A, int n, int mod) {
    vector<int> A_cut(A.begin(), A.begin() + min((int)A.size(), n));
    vector<int> der = poly_deriv(A_cut, mod);
    vector<int> inv = poly_inv(A_cut, n, mod);

    vector<int> prod = mtt(der, inv, mod);
    // 积分后长度恢复为 n
    prod.resize(n - 1);
    return poly_integ(prod, mod);
}

// 多项式指数 (Exp)
// B_{k} = B_{k-1} * (1 - Ln(B_{k-1}) + A)
// 要求 A[0] = 0
vector<int> poly_exp(const vector<int>& A, int n, int mod) {
    if (n == 1) {
        return {1}; // Exp(0) = 1
    }

    vector<int> B = poly_exp(A, (n + 1) / 2, mod);
    B.resize(n); // 扩展空间

    // LnB = Ln(B) mod x^n
    vector<int> LnB = poly_ln(B, n, mod);

    // Calculation: A - LnB + 1
    // 注意：A 可能比 n 短，也可能比 n 长，只取前 n 项
    vector<int> Term(n);
    for (int i = 0; i < n; ++i) {
        int valA = (i < A.size()) ? A[i] : 0;
        int valL = LnB[i];
        Term[i] = (valA - valL + mod) % mod;
    }
    Term[0] = (Term[0] + 1) % mod;

    // B = B * Term
```

```
    vector<int> res = mtt(B, Term, mod);
    res.resize(n);
    return res;
}

// ====================== 主函数示例 ======================

int main() {
    // 优化 I/O
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // 示例输入: n (项数), mod (模数)
    // 计算 e^A(x) mod x^n mod mod
    int n, p;
    if (!(cin >> n >> p)) return 0;

    vector<int> A(n);
    for (int i = 0; i < n; ++i) {
        cin >> A[i];
    }

    // A[0] 必须为 0 才能做 Exp
    // 如果题目不保证，通常需要在数学上做预处理或无解
    assert(A[0] == 0);

    vector<int> ans = poly_exp(A, n, p);

    for (int i = 0; i < n; ++i) {
        cout << ans[i] << (i == n - 1 ? "" : " ");
    }
    cout << endl;

    return 0;
}
```

代码要点说明

1. **模数处理 (`p` vs `P1,P2,P3`)**:
    - 在 `poly_inv`, `poly_ln`, `poly_exp` 等函数内部，加减法和求导积分都是在目标模数 `mod` (即题目给的 p) 下进行的。
    - 唯独在 `mtt` 函数中，我们将系数转移到 $P1, P2, P3$ 下做卷积，通过 CRT 还原出极大值，再对 `mod` 取模。这是保证系数不溢出且结果正确的关键。
2. **截断与 Resize**:
    - 多项式全家桶算法都是基于倍增的。在计算 `poly_exp(n)` 时，会先递归计算 `poly_exp(n/2)`。

- 在每一层迭代中，中间产生的卷积结果（如 $A \times B$）可能会达到 $2n$ 的长度，必须及时用 `.resize(n)` 截断，否则不仅结果错误（保留了高次项干扰后续 CRT），而且时间复杂度会退化。

3. **常数优化**：
   - 代码中将 `NTT_Solver` 实例化为全局变量，避免每次调用 `mtt` 时重新计算原根逆元和重新分配内存。
   - `mtt` 内部的 `vector` 重新分配仍然存在开销。在极端卡常的情况下，可以改用预分配的全局数组或指针操作，但在 $N = 10^5$ 且时限 2s+ 的环境下，上述 `vector` 版本通常可以通过。
4. **积分与逆元**：
   - `poly_integ` 用到了 `modInverse(i + 1, mod)`。这要求 `mod` 是质数（或至少与 $1 \dots N$ 互质）。如果是任意合数模数，这里可能无解，需要扩展欧几里得或者题目保证有解。一般"任意模数 MTT"题目给的模数也是 $10^9 + 7$ 这种质数，只是单纯不是 NTT 模数而已。

三模数 NTT 虽然精度完美，但确实需要跑 9 次 NTT 变换（3个模数 $\times$ (2次正变换+1次逆变换)），常数很大。

要"快"且支持"任意模数"，竞赛中最常用的方案是 **拆系数 FFT (MFT)**。

优化方案：拆系数 FFT (Split-Radix / Coefficient Splitting FFT)

原理：
利用 `double`（或 `long double`）进行复数运算。将系数拆分为高 15 位和低 15 位（$x = a \cdot 2^{15} + b$），利用 FFT 求解。
优势：
只需要 **4 次 FFT** 变换（2 次正变换 + 2 次逆变换）。
相比三模数 NTT（9 次变换），理论速度快 **2 倍以上**。

注意：此方法利用浮点数，对于模数在 10^9 级别、长度 10^5 级别的题目精度足够。

极速版 C++ 模板

为了追求速度，这份代码做了以下优化：

1. **手写 Complex 类**：比 `std::complex` 快。
2. **预处理单位根**：避免在 FFT 循环中频繁调用 `sin/cos`。
3. **合并 DFT**：利用共轭复数性质，将两个多项式的 DFT 合并计算。
4. **合并 IDFT**：一次 IDFT 算出两个部分的卷积结果。

```
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>
#include <iomanip>
```

```cpp
using namespace std;

using i64 = long long;
// 简单的复数结构体，比 std::complex 快
struct Complex {
    double x, y;
    Complex(double x = 0, double y = 0) : x(x), y(y) {}
    Complex operator+(const Complex& b) const { return Complex(x + b.x, y + b.y); }
    Complex operator-(const Complex& b) const { return Complex(x - b.x, y - b.y); }
    Complex operator*(const Complex& b) const { return Complex(x * b.x - y * b.y, x * b.y + y * b.x); }
    Complex conj() const { return Complex(x, -y); }
};

const double PI = acos(-1.0);
const int MAXN = 262144 + 10; // 2^18，根据题目最大长度调整 (例如 1e5 * 2 -> 2^18)

Complex w[MAXN];
int rev[MAXN];
int last_limit = -1;

// 预处理单位根和翻转数组
void init_fft(int limit) {
    if (limit == last_limit) return;
    for (int i = 0; i < limit; i++) {
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) ? (limit >> 1) : 0);
    }
    for (int i = 0; i < limit / 2; i++) {
        w[i] = Complex(cos(PI / (limit / 2) * i), sin(PI / (limit / 2) * i));
    }
    last_limit = limit;
}

// FFT 主体
void fft(vector<Complex>& a, int limit, int type) {
    for (int i = 0; i < limit; i++) {
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    }
    for (int mid = 1; mid < limit; mid <<= 1) {
        // 这里的步长优化是预处理的关键
        int step = (limit >> 1) / mid;
        for (int R = mid << 1, j = 0; j < limit; j += R) {
            for (int k = 0; k < mid; k++) {
                // type=1 时用预处理的 w，type=-1 时用 w 的共轭 (im 部分取反)
                Complex buf = w[k * step];
                if (type == -1) buf.y = -buf.y;
```

```
                Complex x = a[j + k];
                Complex y = buf * a[j + k + mid];
                a[j + k] = x + y;
                a[j + k + mid] = x - y;
            }
        }
    }
    if (type == -1) {
        for (int i = 0; i < limit; i++) {
            a[i].x /= limit;
            a[i].y /= limit;
        }
    }
}

// 任意模数乘法核心
// 输入: 多项式 A, B 和模数 mod
// 输出: 卷积结果
vector<i64> multiply(const vector<i64>& A, const vector<i64>& B, i64 mod) {
    int n = A.size() - 1;
    int m = B.size() - 1;
    int limit = 1;
    while (limit <= n + m) limit <<= 1;

    init_fft(limit);

    // 拆系数常数，通常取 32768 (2^15)
    // 使得 val = a * M + b，其中 a,b < 32768
    // 这样 a*a 最大 10^9，fit in double
    const int M = 32768;

    // 准备复数数组
    // P 存放 A 的拆分: real=A_low, imag=A_high
    // Q 存放 B 的拆分: real=B_low, imag=B_high
    vector<Complex> P(limit), Q(limit);

    for (int i = 0; i <= n; i++) P[i] = Complex(A[i] % M, A[i] / M);
    for (int i = 0; i <= m; i++) Q[i] = Complex(B[i] % M, B[i] / M);

    // 1. 两次正变换
    fft(P, limit, 1);
    fft(Q, limit, 1);

    // 准备逆变换数组
    // R1 存放 A_low * B_low + i * A_high * B_high
    // R2 存放 A_low * B_high + A_high * B_low
    vector<Complex> R1(limit), R2(limit);

    for (int i = 0; i < limit; i++) {
        // 提取 DFT 后的分量
        // 如果 F(k) 是 f(t) 的 DFT
        // 且 f(t) = a(t) + i*b(t) (a,b 为实数信号)
```

```
        // 那么 DFT(a)(k) = 0.5 * (F(k) + conj(F(N-k)))
        //      DFT(b)(k) = -0.5i * (F(k) - conj(F(N-k)))

        int j = (limit - i) & (limit - 1); // j = (N-i) % N

        Complex da = P[i];
        Complex db = P[j].conj();
        Complex Al = (da + db) * Complex(0.5, 0);        // DFT(A_low)
        Complex Ah = (da - db) * Complex(0, -0.5);       // DFT(A_high)

        Complex dc = Q[i];
        Complex dd = Q[j].conj();
        Complex Bl = (dc + dd) * Complex(0.5, 0);        // DFT(B_low)
        Complex Bh = (dc - dd) * Complex(0, -0.5);       // DFT(B_high)

        // 构造待逆变换的多项式
        // 我们需要计算：
        // ans = AlBl + M(AlBh + AhBl) + M^2(AhBh)

        // 技巧：
        // 将 AlBl (实部) 和 AhBh (虚部) 放在一个复数里做一次 IDFT
        R1[i] = Al * Bl + (Ah * Bh) * Complex(0, 1);

        // 将交叉项放在另一个里
        R2[i] = Al * Bh + Ah * Bl;
    }

    // 2. 两次逆变换
    fft(R1, limit, -1);
    fft(R2, limit, -1);

    vector<i64> ans(n + m + 1);
    for (int i = 0; i <= n + m; i++) {
        // 提取结果并取模
        // R1.x -> AlBl
        // R1.y -> AhBh
        // R2.x -> AlBh + AhBl

        // 必须使用 round 修正浮点误差
        i64 i_ll = (i64)(R1[i].x + 0.5) % mod;
        i64 i_hh = (i64)(R1[i].y + 0.5) % mod;
        i64 i_lh = (i64)(R2[i].x + 0.5) % mod;

        // 最终结果 = ll + (lh * M) + (hh * M^2)
        // 使用 __int128 防止中间过程溢出 long long
        // (虽然 M*M*hh 可能到 10^18，但加起来可能溢出)
        __int128 res = i_ll;
        res = (res + (__int128)i_lh * M) % mod;
        res = (res + (__int128)i_hh * M * M) % mod;

        ans[i] = (i64)res;
    }
```

```
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    i64 p;
    if (cin >> n >> m >> p) {
        vector<i64> A(n + 1), B(m + 1);
        for (int i = 0; i <= n; i++) cin >> A[i];
        for (int i = 0; i <= m; i++) cin >> B[i];

        vector<i64> res = multiply(A, B, p);

        for (int i = 0; i < res.size(); i++) {
            cout << res[i] << (i == res.size() - 1 ? "" : " ");
        }
        cout << "\n";
    }
    return 0;
}
```

代码分析：为什么这个更快?

1. **减少变换次数**:
    - **三模数 NTT**: 3次正变换 + 1次逆变换 = 4次，但是要对3个模数分别做，总共**12次变换**（或者优化后9次）。
    - **此模板**:
        - 将 A 的高低位打包进一个复数 P。
        - 将 B 的高低位打包进一个复数 Q。
        - 2次正变换：$P \to DFT(P), Q \to DFT(Q)$。
        - 频率域操作：$O(N)$。
        - 将结果打包回 $R1, R2$。
        - 2次逆变换。
        - **总共仅 4 次变换。**
2. **内存友好**:
    - 复数结构体 `struct Complex` 是 Plain Old Data (POD)，内存连续，缓存命中率比 STL 的高。
    - 预处理 `w` 数组，避免了昂贵的 `sin/cos` 重复计算。
3. **适用范围**:
    - 模数 $P \leq 10^{14}$ 左右都能保证精度（中间值在 `double` 的 53bit 有效数字内）。

- 对于一般的竞赛题目（模数 10^9+7 或 10^9+9），这绝对是首选。

使用提示

- **MAXN**: 如果你的 N, M 高达 10^5，卷积后长度可能达到 2×10^5，所以 `MAXN` 最好设为 $2^{18} \approx 26$万 或者更大。
- **Long Long**: 已经全面使用 `i64`，最后的合并使用了 `__int128` 确保绝对安全。

这是一个非常硬核的需求。在任意模数（MTT）的基础上实现多项式全家桶（求逆、对数、指数），计算量会非常大，因为每层递归都要跑几次 4-FFT 的 MTT。

为了保证正确性，必须假设**模数 `mod` 是一个质数**（或者至少能保证所需系数存在逆元），因为求导、积分和求逆都需要用到模逆元。

以下是在你提供的板子基础上，增加了 `poly_inv` (求逆), `poly_ln` (对数), `poly_exp` (指数) 的完整代码。

核心变更点

1. 增加了 **快速幂** 和 **模逆元** 函数。
2. 增加了 **多项式求导** 和 **多项式积分**。
3. 实现了 **多项式求逆 (Newton Iteration)**: $G \equiv G_0(2 - AG_0) \pmod{x^n}$。
4. 实现了 **多项式对数**: $\ln A = \int (A' A^{-1}) dx$。
5. 实现了 **多项式指数**: $F \equiv F_0(1 - \ln F_0 + A) \pmod{x^n}$。

完整代码

```
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>
#include <iomanip>
#include <cassert>

using namespace std;

using i64 = long long;

// ================== 基础数学工具 ==================

// 快速幂
i64 qpow(i64 a, i64 b, i64 p) {
    i64 res = 1;
    for (; b; b >>= 1, a = (__int128)a * a % p) {
        if (b & 1) res = (__int128)res * a % p;
    }
    return res;
```

```
}

// 模逆元 (前提: mod 是质数)
i64 modInverse(i64 n, i64 p) {
    return qpow(n, p - 2, p);
}

// ================== FFT / MTT 核心部分 ==================

struct Complex {
    double x, y;
    Complex(double x = 0, double y = 0) : x(x), y(y) {}
    Complex operator+(const Complex& b) const { return Complex(x + b.x, y + b.y); }
    Complex operator-(const Complex& b) const { return Complex(x - b.x, y - b.y); }
    Complex operator*(const Complex& b) const { return Complex(x * b.x - y * b.y, x
* b.y + y * b.x); }
    Complex conj() const { return Complex(x, -y); }
};

const double PI = acos(-1.0);
// 注意：做 exp/inv 时，中间过程多项式长度可能翻倍，MAXN 需要开大一点
// 如果题目 N=1e5，exp 过程可能需要处理到 2^18，保险起见开到 2^19 或 2^20
const int MAXN = 524288 + 10;

Complex w[MAXN];
int rev[MAXN];
int last_limit = -1;

void init_fft(int limit) {
    if (limit == last_limit) return;
    for (int i = 0; i < limit; i++) {
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) ? (limit >> 1) : 0);
    }
    for (int i = 0; i < limit / 2; i++) {
        w[i] = Complex(cos(PI / (limit / 2) * i), sin(PI / (limit / 2) * i));
    }
    last_limit = limit;
}

void fft(vector<Complex>& a, int limit, int type) {
    for (int i = 0; i < limit; i++) {
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    }
    for (int mid = 1; mid < limit; mid <<= 1) {
        int step = (limit >> 1) / mid;
        for (int R = mid << 1, j = 0; j < limit; j += R) {
            for (int k = 0; k < mid; k++) {
                Complex buf = w[k * step];
                if (type == -1) buf.y = -buf.y;
                Complex x = a[j + k];
                Complex y = buf * a[j + k + mid];
                a[j + k] = x + y;
```

```
                a[j + k + mid] = x - y;
            }
        }
    }
    if (type == -1) {
        for (int i = 0; i < limit; i++) {
            a[i].x /= limit;
            a[i].y /= limit;
        }
    }
}

// 任意模数乘法 MTT
vector<i64> multiply(const vector<i64>& A, const vector<i64>& B, i64 mod) {
    int n = A.size() - 1;
    int m = B.size() - 1;
    int limit = 1;
    while (limit <= n + m) limit <<= 1;

    init_fft(limit);

    const int M = 32768;
    vector<Complex> P(limit), Q(limit);

    for (int i = 0; i <= n; i++) P[i] = Complex(A[i] % M, A[i] / M);
    for (int i = 0; i <= m; i++) Q[i] = Complex(B[i] % M, B[i] / M);

    fft(P, limit, 1);
    fft(Q, limit, 1);

    vector<Complex> R1(limit), R2(limit);
    for (int i = 0; i < limit; i++) {
        int j = (limit - i) & (limit - 1);
        Complex da = P[i], db = P[j].conj();
        Complex Al = (da + db) * Complex(0.5, 0);
        Complex Ah = (da - db) * Complex(0, -0.5);
        Complex dc = Q[i], dd = Q[j].conj();
        Complex Bl = (dc + dd) * Complex(0.5, 0);
        Complex Bh = (dc - dd) * Complex(0, -0.5);

        R1[i] = Al * Bl + (Ah * Bh) * Complex(0, 1);
        R2[i] = Al * Bh + Ah * Bl;
    }

    fft(R1, limit, -1);
    fft(R2, limit, -1);

    vector<i64> ans(n + m + 1);
    for (int i = 0; i <= n + m; i++) {
        i64 i_ll = (i64)(R1[i].x + 0.5) % mod;
        i64 i_hh = (i64)(R1[i].y + 0.5) % mod;
        i64 i_lh = (i64)(R2[i].x + 0.5) % mod;
```

```
        __int128 res = i_ll;
        res = (res + (__int128)i_lh * M) % mod;
        res = (res + (__int128)i_hh * M * M) % mod;
        ans[i] = (i64)res;
    }
    return ans;
}

// ================== 多项式全家桶 ==================

// 多项式求导
vector<i64> poly_deriv(const vector<i64>& A, i64 mod) {
    if (A.size() <= 1) return {0};
    vector<i64> res(A.size() - 1);
    for (int i = 1; i < A.size(); i++) {
        res[i - 1] = A[i] * i % mod;
    }
    return res;
}

// 多项式积分
vector<i64> poly_integ(const vector<i64>& A, i64 mod) {
    vector<i64> res(A.size() + 1);
    res[0] = 0;
    for (int i = 0; i < A.size(); i++) {
        res[i + 1] = A[i] * modInverse(i + 1, mod) % mod;
    }
    return res;
}

// 多项式求逆 (mod x^n)
// G_new = G_old * (2 - A * G_old)
vector<i64> poly_inv(const vector<i64>& A, int n, i64 mod) {
    if (n == 1) {
        return {modInverse(A[0], mod)};
    }
    // 递归求解 (n+1)/2
    vector<i64> B = poly_inv(A, (n + 1) / 2, mod);

    // 准备当前层的 A (截取前 n 项)
    vector<i64> tmpA(A.begin(), A.begin() + min((int)A.size(), n));

    // 计算 A * B
    // 注意：这里我们只需要模 x^n 的结果，multiply 可能会产生更长的
    vector<i64> AB = multiply(tmpA, B, mod);
    if (AB.size() > n) AB.resize(n); // 截断

    // 计算 B * (2 - AB)
    // 实际上就是 2B - B * AB
    // 为了利用 multiply 已经算出的 AB，我们把公式变一下：
    // res = 2B - B * AB
    // 但为了使用卷积，我们计算 B * (2 - AB)。
```

```
    // 构造 (2 - AB) 多项式
    vector<i64> TwoMinusAB(n);
    for(int i=0; i<n; ++i) {
        i64 val = (i < AB.size() ? AB[i] : 0);
        if (i == 0) {
            TwoMinusAB[i] = (2 - val + mod) % mod;
        } else {
            TwoMinusAB[i] = (mod - val) % mod;
        }
    }

    vector<i64> res = multiply(B, TwoMinusAB, mod);
    if (res.size() > n) res.resize(n);
    return res;
}

// 多项式对数 (mod x^n)
// ln A = integ( deriv(A) * inv(A) )
// 前提: A[0] 必须为 1
vector<i64> poly_ln(const vector<i64>& A, int n, i64 mod) {
    assert(A[0] == 1); // 基本定义要求
    vector<i64> der = poly_deriv(A, mod);
    // 对 A 截断到 n 进行求逆
    vector<i64> tmpA = A;
    if(tmpA.size() > n) tmpA.resize(n);
    vector<i64> invA = poly_inv(tmpA, n, mod);

    vector<i64> prod = multiply(der, invA, mod);
    // 积完分后的长度应该是 n
    if (prod.size() > n - 1) prod.resize(n - 1);

    return poly_integ(prod, mod);
}

// 多项式指数 (mod x^n)
// F_new = F_old * (1 - ln(F_old) + A)
// 前提: A[0] 必须为 0
vector<i64> poly_exp(const vector<i64>& A, int n, i64 mod) {
    if (n == 1) {
        return {1}; // exp(0) = 1
    }
    vector<i64> B = poly_exp(A, (n + 1) / 2, mod);
    B.resize(n); // 扩展大小以便后续计算

    // 计算 ln(B)
    vector<i64> lnB = poly_ln(B, n, mod);

    // 构造 P = (1 - lnB + A)
    vector<i64> P(n);
    for (int i = 0; i < n; i++) {
        i64 a_val = (i < A.size() ? A[i] : 0);
        i64 l_val = (i < lnB.size() ? lnB[i] : 0);
```

```
        // P[i] = A[i] - lnB[i]
        P[i] = (a_val - l_val + mod) % mod;
    }
    // +1 (常数项)
    P[0] = (P[0] + 1) % mod;

    vector<i64> res = multiply(B, P, mod);
    if (res.size() > n) res.resize(n);
    return res;
}

// ================== 主函数 ==================

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    i64 p;
    // 输入格式: 长度n 模数p (通常p需要是质数)
    if (cin >> n >> p) {
        vector<i64> A(n);
        for (int i = 0; i < n; i++) cin >> A[i];

        // 计算 Exp(A) mod x^n
        // 题目保证 A[0] = 0
        vector<i64> res = poly_exp(A, n, p);

        for (int i = 0; i < n; i++) {
            cout << res[i] << (i == n - 1 ? "" : " ");
        }
        cout << "\n";
    }
    return 0;
}
```

使用注意

1. **模数 `p` 的要求**：`poly_exp` 依赖 `poly_ln`，后者依赖 `poly_deriv/integ/inv`，这些步骤中会用到除法（乘以逆元）。因此，**模数 `p` 必须是质数**（或者至少保证 $1 \sim n$ 的所有数在模 `p` 下都有逆元）。如果 `p` 不是质数，标准的求导/积分/求逆逻辑会失效。
2. **常数项限制**：
    - `poly_ln(A)` 要求 $A[0] = 1$。
    - `poly_exp(A)` 要求 $A[0] = 0$（因为结果的常数项必然是 $e^0 = 1$）。
3. **复杂度与常数**：
    - `poly_exp` 的时间复杂度是 $O(N \log N)$。
    - 但是常数非常大。MTT 本身做一次乘法需要 4 次 FFT，`poly_exp` 调用 `poly_ln`，`poly_ln` 调用 `poly_inv`，层层嵌套，每一层牛顿迭代都包含乘法。

    - 在 $N = 10^5$ 时，可能会跑几秒钟（取决于机器），这已经是任意模数下的极限优化了（如果不手写 AVX 指令集优化的话）。
4. **MAXN 设置**：代码中设为 2^{19} (524288)。这是为了应对输入 N 接近 10^5 甚至 2×10^5 的情况。FFT 长度必须是 2 的幂次且大于 $N + M$。